

11.2 Training Dynamics

Minimax J. Non-saturating $-\log D(G(z))$. A 99% D is often a dead G.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

The two nets have opposite goals. You cannot treat a GAN as one ordinary loss. Each step is a small fight: first improve D , then improve G , and try not to let either win too hard.

First time this training loop appears. GAN **training** is a min-max game with two phases per iteration.

What. D climbs to tell real from fake. G climbs to fool D . The original saturating loss for G is weak; people use $-\log D(G(z))$. **Why.** If you only train D to 99%, G gets no gradient. If you only train G , D is stale. **Architecture.** Same two nets. Alternate (or simultaneous) SGD steps. No KL term here; that is RLHF. **How.** Phase D: maximize $\log D$ on reals + $\log(1 - D)$ on fakes. Phase G: non-saturating $-\log D(G(z))$. **Formula.** $J(D, G) = \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$. G minimizes a surrogate, not always this J . **Tradeoffs.** + When balanced, samples improve. – Oscillation, vanishing gradients for G , sensitivity to step sizes; next note is collapse.

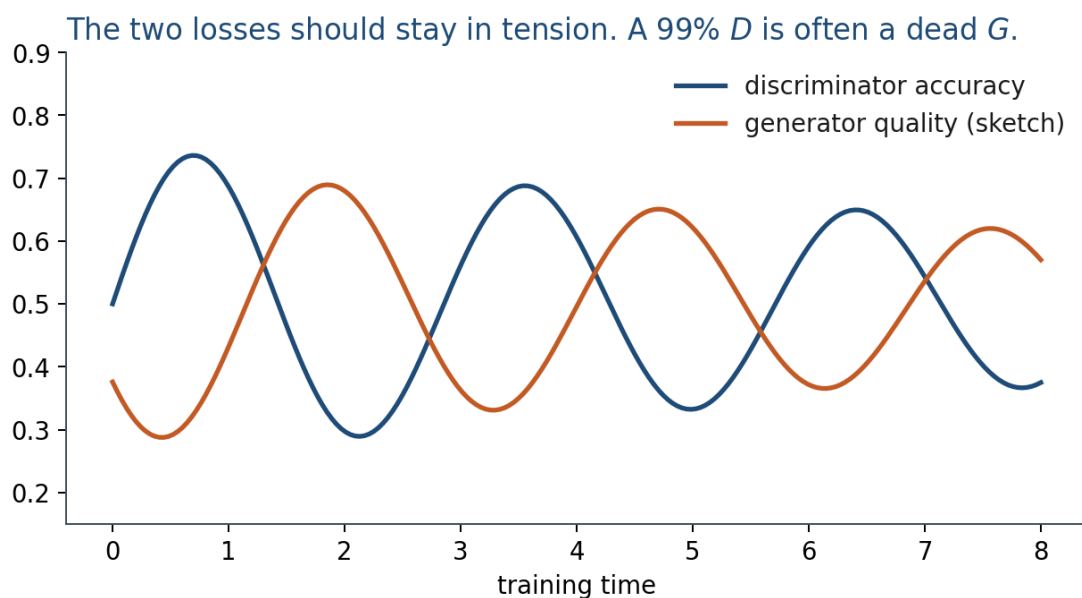
1. The min-max game

Goodfellow et al. write a two-player objective

$$\min_{\theta_g} \max_{\theta_d} J(\theta_g, \theta_d),$$

$$J = \mathbb{E}_{x \sim p_r} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] .$$

D wants J large (correct labels). G wants J small (fakes that look real). A Nash point you *hope* for: G matches p_r , and D is a coin flip. Training does not guarantee you land there.



Read the two terms out loud. First term: on reals, $\log D(x)$ is large when $D \approx 1$. Second term: on fakes, $\log(1 - D(G(z)))$ is large when $D \approx 0$. G only appears in the second term, and it wants that term small, i.e. $D(G(z))$ near 1.

2. Two phases per iteration

1. **Train** D . A batch of reals (label 1) and fakes from the current G (label 0). Binary cross-entropy. Update **only** θ_d .
2. **Train** G . Fresh noise, fresh fakes. Labels are all 1: you want D to be *wrong*. Freeze θ_d ; update **only** θ_g .

In code you often maximize $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$. Early in training D is too good and the second term saturates: if $D(G(z)) \approx 0$, then $\log(1 - D) \approx 0$ and the slope w.r.t. G is tiny. $-\log D(G(z))$ still has slope.

Lab 11's loop is exactly this, with `BCEWithLogitsLoss`: one D step, one G step, 400 iterations. Print both losses. Do not rank checkpoints by the smaller number.

3. The tension

If D is too weak, G gets a noisy, lying teacher. If D is too strong, G 's gradient vanishes. The two parameter vectors can **oscillate**: losses look fine, then the samples jump. Learning rates, how many D steps per G step, and batch size all move that balance.

Watching J is not evaluation. A falling generator loss can mean better fakes *or* a collapsing D . You still have to look at samples and at **coverage** (note 11.3). Lab 11 is a 2-D version of this fight on purpose: you will see the tension in a scatter plot, not in a single scalar.

4. Teaching this note

About **35 minutes** at the board, then **~10 minutes** of video.

- **0–12 min.** Write J and mark who maximizes which term. Circle that G does not appear in the real term.
 - **12–22 min.** The two-phase loop. Non-saturating trick: $\max_G \log D(G(z))$.
 - **22–33 min.** Worked numbers: one real, one fake, both forms of the G loss.
 - **Then** play CS231N **54:00–64:00** (minimax objective and the training algorithm). Pause on the “train D then G ” slide.
-

5. Worked example

One real x with $D(x) = 0.9$, one fake with $D(G(z)) = 0.2$. Use natural log, $\log 0.9 \approx -0.105$, $\log 0.8 \approx -0.223$, $\log 0.2 \approx -1.609$.

Discriminator’s sample of J :

$$\log D(x) + \log(1 - D(G(z))) = \log 0.9 + \log 0.8 \approx -0.105 - 0.223 = -0.328.$$

D wants this **larger** (less negative): push $D(x)$ up, push $D(G(z))$ down.

Saturating generator loss (the J term G minimizes):

$$\log(1 - D(G(z))) = \log 0.8 \approx -0.223.$$

If instead $D(G(z)) = 0.01$, that becomes $\log 0.99 \approx -0.010$: almost flat. The non-saturating surrogate is $-\log D(G(z))$. At $D = 0.2$ that is $-\log 0.2 \approx 1.609$; at $D = 0.01$ it is $-\log 0.01 \approx 4.605$. Still a slope. That is why Lab 11 feeds **ones** into the generator’s BCE: it is the non-saturating form.

 A sample of $\log(1 - D(G(z)))$

J , then a flat saturating G loss" src="graphics/11.2-gan-training/j-numeric.png" />

Stanford CS231N 2025 L13 writes the same minimax and the practical loop: update D , then G . A 99% discriminator is the saturating regime, not a trophy.

6. Where students get stuck

- Updating both nets in one `loss.backward()` on a shared graph. Detach the fakes on the D step; freeze D on the G step.
 - Reading a falling G loss as “better pictures.” It can mean D got worse.
 - Using $\log(1 - D(G(z)))$ early, when D is already sharp, and concluding “GANs cannot train.”
-

7. Video

Watch [Stanford CS231N 2017 lecture 13, Generative Models](#), 54:00–64:00.

Pause on the minimax J and on the practical loop (alternate D and G). Same URL as note **11.1**; do not replay 46:45–54:00 unless someone missed it.

8. Practice

1. In phase 2 the fake labels are 1. Who is being lied to, and why is that the generator's loss?
2. Why might two discriminator steps per generator step help early training, and how could it hurt later?
3. The generator loss dropped and the pictures got worse. Give one explanation that is not "the learning rate is wrong."
4. $D(G(z)) = 0.05$. Compute $\log(1 - D(G(z)))$ and $-\log D(G(z))$ (natural log; $\log 0.95 \approx -0.051$, $\log 0.05 \approx -3.00$). Which number still has room to move G ?
5. A batch has $D(x) = 0.8$ on every real and $D(G(z)) = 0.4$ on every fake. Write the one-sample J as a number ($\log 0.8 \approx -0.223$, $\log 0.6 \approx -0.511$).